

Phase 6: REST API Mastery (2 Weeks)

Goal of This Phase

By the end of these two weeks, you will learn how to build professional REST APIs like backend developers do in real-world projects.

You will understand:

- How to create clean API endpoints
 - How to send and receive data correctly
 - How to return proper status codes
 - How to search, filter, sort, and paginate data
 - How to handle errors professionally
 - How to build a complete Product Management API
-

Week 1 – Learn REST API Fundamentals

1. HTTP Methods

Learn what each HTTP method does and when to use it.

GET

Used to **fetch data** from the database.

Example:

```
GET /products
```

Returns all products.

POST

Used to **create new data**.

Example:

```
POST /products
```

Creates a new product.

PUT

Used to **replace an entire document**.

If you don't send a field, it may be removed or replaced.

Example:

```
PUT /products/:id
```

PATCH

Used to **update only specific fields**.

Example:

```
PATCH /products/:id
```

Update only the price or stock without changing the other fields.

DELETE

Used to **remove data**.

Example:

```
DELETE /products/:id
```

Deletes the product.

Practice

Make sure you understand:

- Difference between PUT and PATCH
 - What is an Idempotent Method?
 - What is a Safe Method?
-

2. HTTP Status Codes

Every API response should return the correct status code.

Success Responses

200 OK

Request completed successfully.

Example:

```
GET /products
```

201 Created

A new resource was created.

Example:

```
POST /products
```

202 Accepted

The request has been accepted and will be processed later.

204 No Content

Request was successful but nothing needs to be returned.

Usually used after deleting data.

Client Error Responses

400 Bad Request

The request contains invalid data.

Example:

Missing required fields.

401 Unauthorized

The user is not logged in.

403 Forbidden

The user is logged in but doesn't have permission.

404 Not Found

Requested resource doesn't exist.

Example:

Product ID not found.

409 Conflict

The request conflicts with existing data.

Example:

Trying to create a category with an existing name.

422 Unprocessable Entity

Data format is correct but validation fails.

Example:

Negative price.

Server Errors

500 Internal Server Error

Something went wrong on the server.

502 Bad Gateway

Another server returned an invalid response.

503 Service Unavailable

Server is temporarily unavailable.

Learn when to use each status code correctly.

3. REST API Naming Conventions

Write clean and meaningful API endpoints.

Good Examples

```
GET    /products
GET    /products/:id
POST   /products
PUT    /products/:id
PATCH /products/:id
DELETE /products/:id
```

Avoid using verbs.

Bad Examples

```
/getProducts
/createProduct
/deleteProduct
```

Remember:

The HTTP method already tells what action is happening.

4. Request and Response Design

Understand the different parts of a request.

Request Body

Data sent when creating or updating resources.

Example:

```
POST /products
```

```
{
  "name": "iPhone",
```

```
"price":70000
}
```

Route Parameters

Used to identify one resource.

Example

```
GET /products/123
```

Here **123** is the route parameter.

Query Parameters

Used for filtering, sorting, searching and pagination.

Example

```
GET /products?page=2&limit=10
```

Headers

Used to send extra information.

Examples:

- Content-Type
 - Authorization
 - Accept
-

API Response Format

Keep every response consistent.

Example

```
{
  "success": true,
  "message": "Product created successfully",
}
```

```
"data": {}  
}
```

Error Example

```
{  
  "success": false,  
  "message": "Product not found"  
}
```

5. Query Parameters

Learn how to use query parameters for:

- Searching
- Pagination
- Sorting
- Filtering

Examples

```
GET /products?page=2
```

```
GET /products?limit=10
```

```
GET /products?sort=price
```

```
GET /products?category=mobile
```

```
GET /products?search=iphone
```

6. Pagination

Instead of sending all products, send only a small number.

Learn:

- page
- limit

- skip

Example

```
GET /products?page=2&limit=10
```

Example Response

```
{
  "page": 2,
  "limit": 10,
  "totalPages": 15,
  "totalProducts": 145,
  "data": []
}
```

7. Sorting

Allow users to order data.

Examples

```
Price (Low → High)

Price (High → Low)

Newest Products

Oldest Products

Highest Rated
```

API Examples

```
GET /products?sort=price
```

```
GET /products?sort=-price
```

8. Filtering

Return only matching products.

Filter by:

- Category
- Brand
- Price Range
- Availability

Examples

```
GET /products?category=Laptop
```

```
GET /products?brand=Apple
```

```
GET /products?priceMin=50000
```

```
GET /products?priceMax=100000
```

Combine filters

```
GET /products?category=Laptop&brand=Dell
```

9. Searching

Allow users to search products.

Search using:

- Product Name
- Category
- Brand

Use:

- Regular Expressions
- MongoDB Text Search

Example

```
GET /products?search=macbook
```

10. Error Handling

Always return meaningful error messages.

Example

```
{
  "success": false,
  "message": "Product not found"
}
```

Handle errors like:

- Invalid Product ID
 - Product Not Found
 - Validation Failed
 - Database Errors
-

Week 2 – Build a Product Management API

Now apply everything you've learned by building a real backend project.

Models

Product

```
name
description
price
category
brand
stock
rating
image
```

Category

```
name
description
```

Features to Build

CRUD Operations

Build APIs to:

- Create Product
 - Get All Products
 - Get Product By ID
 - Update Product
 - Delete Product
-

Pagination

```
GET /products?page=2&limit=10
```

Sorting

```
GET /products?sort=price
```

```
GET /products?sort=-price
```

```
GET /products?sort=rating
```

```
GET /products?sort=createdAt
```

Filtering

```
GET /products?category=Laptop
```

```
GET /products?brand=Apple
```

```
GET /products?priceMin=50000
```

```
GET /products?priceMax=100000
```

Combine multiple filters

```
GET /products?category=Laptop&brand=Dell
```

Search

```
GET /products?search=macbook
```

Validation

Validate user input before saving data.

Examples:

- Name is required
- Price must be greater than 0
- Stock cannot be negative

Error Handling

Handle cases like:

- Invalid Product ID
- Product Not Found
- Validation Errors
- Database Errors

Consistent Response Format

Every API should return the same response structure.

Success

```
{
  "success":true,
  "message":"Product fetched successfully",
  "data":[]
}
```

Error

```
{
  "success":false,
  "message":"Product not found"
}
```

Bonus Features (Optional)

If you finish early, try adding these features.

Categories API

- Create Category
- Get Categories
- Update Category
- Delete Category
- Get Products by Category

Product Reviews

- Add Review
- Update Review
- Delete Review
- Calculate Average Rating

Inventory Management

- Increase Stock
- Decrease Stock
- View Out-of-Stock Products

Soft Delete

Instead of deleting permanently, mark the product as deleted.

```
isDeleted = true
```

API Documentation

Document your APIs using Swagger/OpenAPI.

Environment Variables

Store sensitive values in a `.env` file.

Example

```
PORT
MONGODB_URI
JWT_SECRET
```

What You Will Learn After This Phase

By completing this project, you will be able to:

- ✓ Design RESTful APIs
- ✓ Build complete CRUD APIs
- ✓ Use Route Parameters
- ✓ Use Query Parameters
- ✓ Validate User Input
- ✓ Return Proper HTTP Status Codes
- ✓ Implement Pagination
- ✓ Implement Sorting
- ✓ Implement Filtering
- ✓ Implement Searching
- ✓ Handle Errors Professionally

✓ Write Clean and Consistent API Responses

✓ Document APIs with Swagger

These are the core backend skills used in almost every real-world application, such as e-commerce platforms, blogs, project management systems, and social media apps.

Once you complete this phase confidently, you'll be ready to move on to the next step: **Authentication and Authorization**, where you'll learn how to secure your APIs using JWT, role-based access control, and protected routes.